Dipartimento di Ingegneria Enzo Ferrari
Corso di laurea in Ingegneria Informatica
Università degli Studi di Modena e Reggio Emilia

Creazione e sviluppo dell'applicazione Payper

Relatore:
Nicola Bicocchi

Tesi di Laurea di:
Samuele Mazzi

Introduzione

Start-up per la creazione di scontrini sostenibili

Ce n'è davvero bisogno?

- Gli scontrini non sono riciclabili
- 3 milioni di alberi ogni anno negli USA
- Sostanze tossiche come BPA e BPS

STRUMENTI USATI

Flutter

Framework basato sul linguaggio Dart

STATO

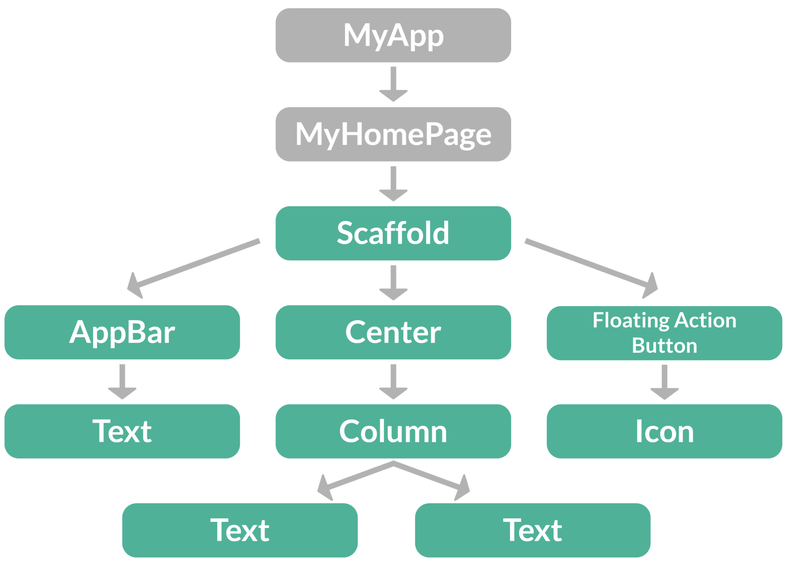
UI = f (state)

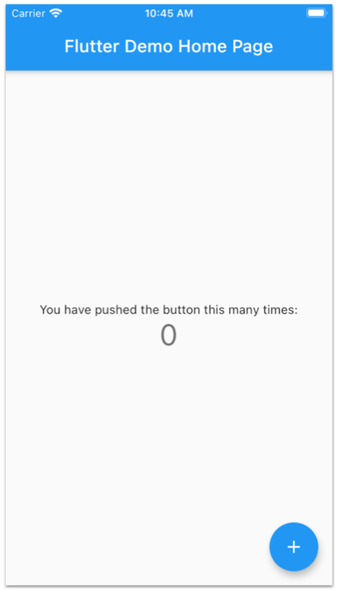
Layout

Build e setState

Variabili di stato

WIDGET





STRUMENTI USATI

Firebase

- **Realtime Database**: NoSQL database, in formato JSON
- **Firestore Database**: NoSQL database, strutturato in documenti e collezioni
- **Cloud Storage**: spazio di archiviazione online
- **Cloud Messaging**: invio di notifiche
- **Cloud Functions**: esecuzione di codice di backend ed API
- **Authentication**: gestione autenticazione
- **Analytics**: raccolta di dati sull'utilizzo

PANORAMICA GENERALE

Overview

- Digitalizzazione
- Catalogazione scontrini
- Sistema di progresso

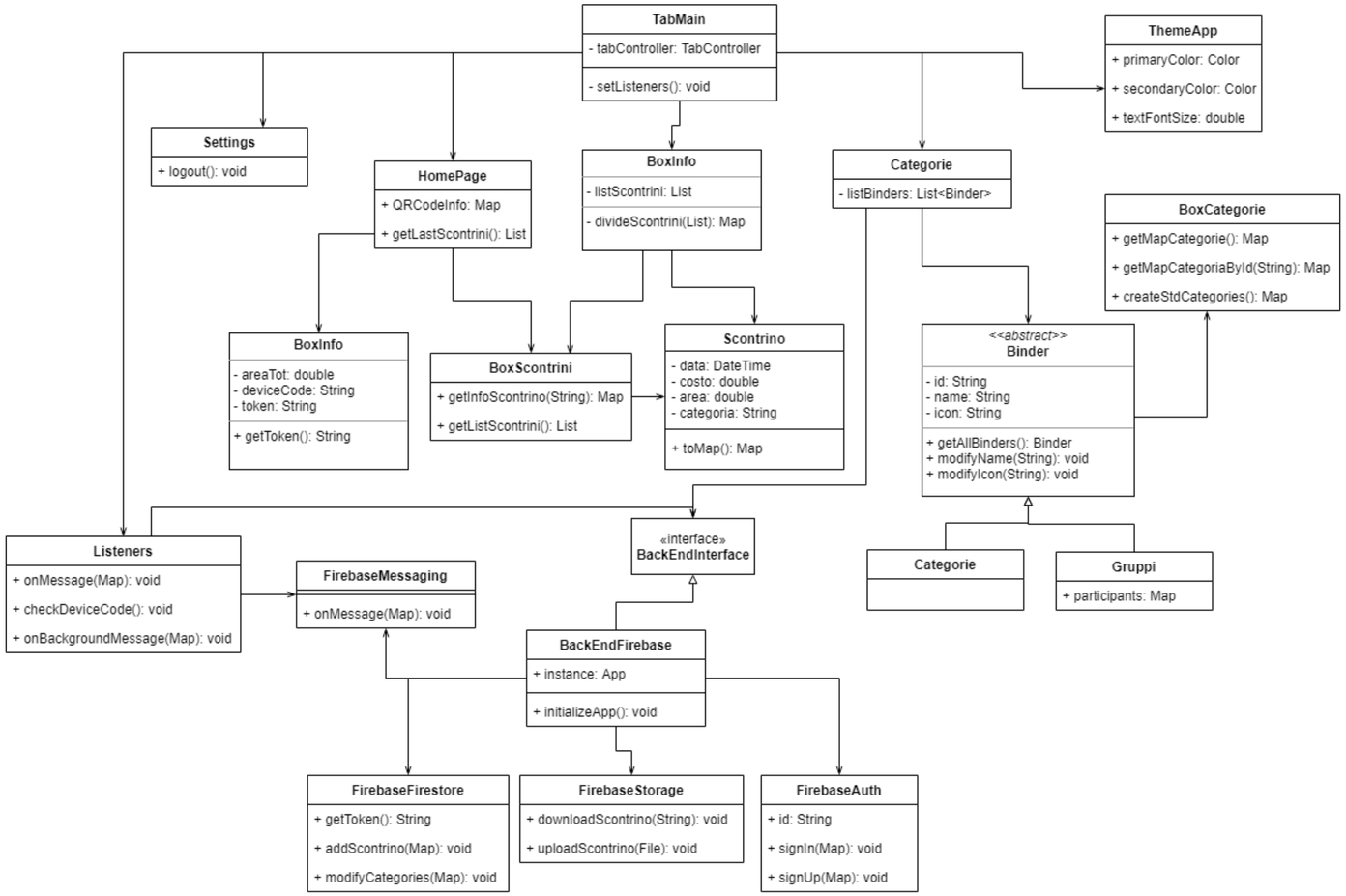
Aggiunta scontrini cartacei

Gestione carte fedeltà

Pubblicità mirate

PANORAMICA GENERALE

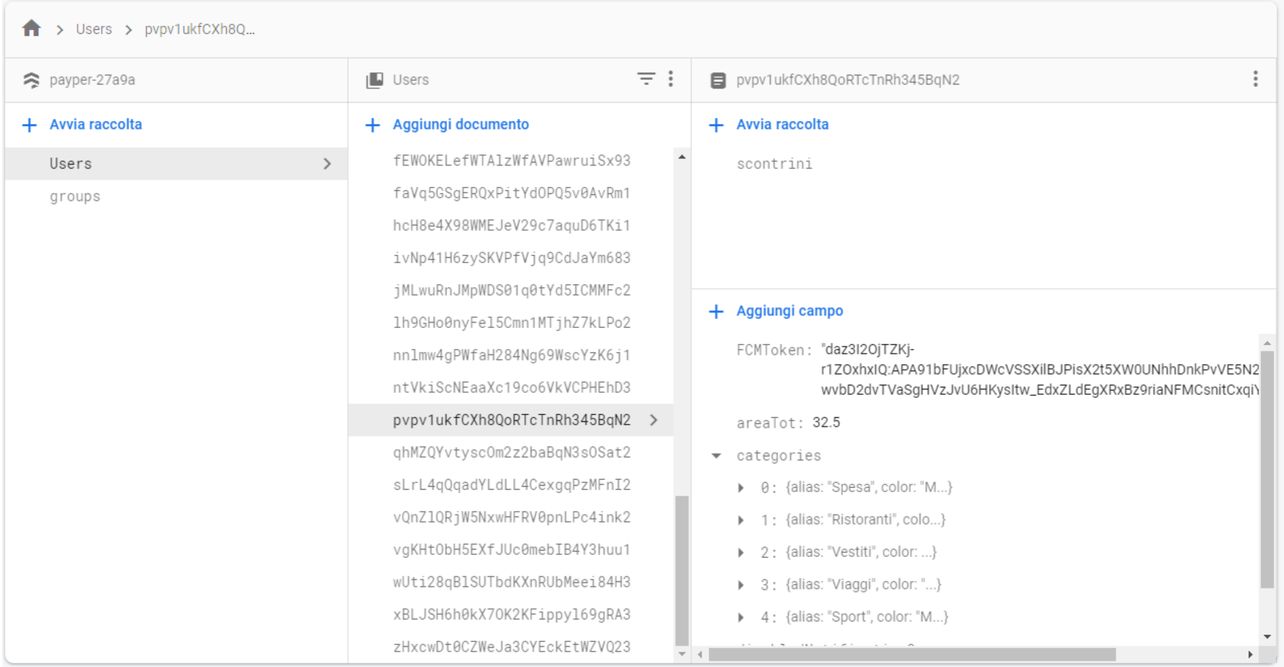
Struttura del software



DETTAGLI

Struttura dei database

FIRESTORE



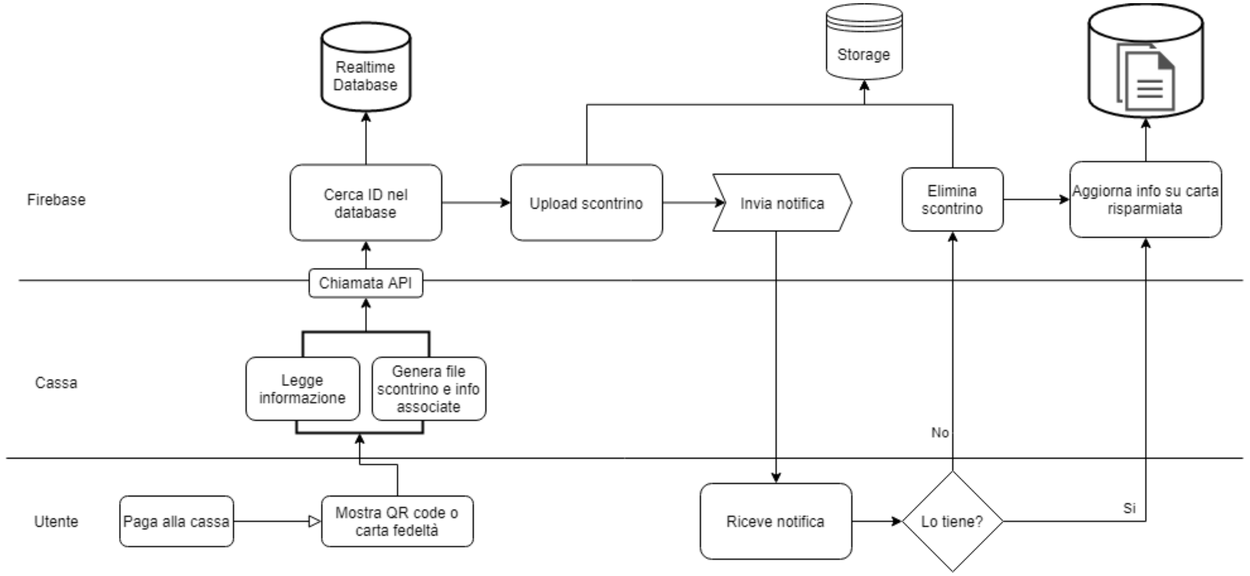
Utenti e categorie condivise

REALTIME

Area scontrini, codici device
e carte fedeltà

DETTAGLI

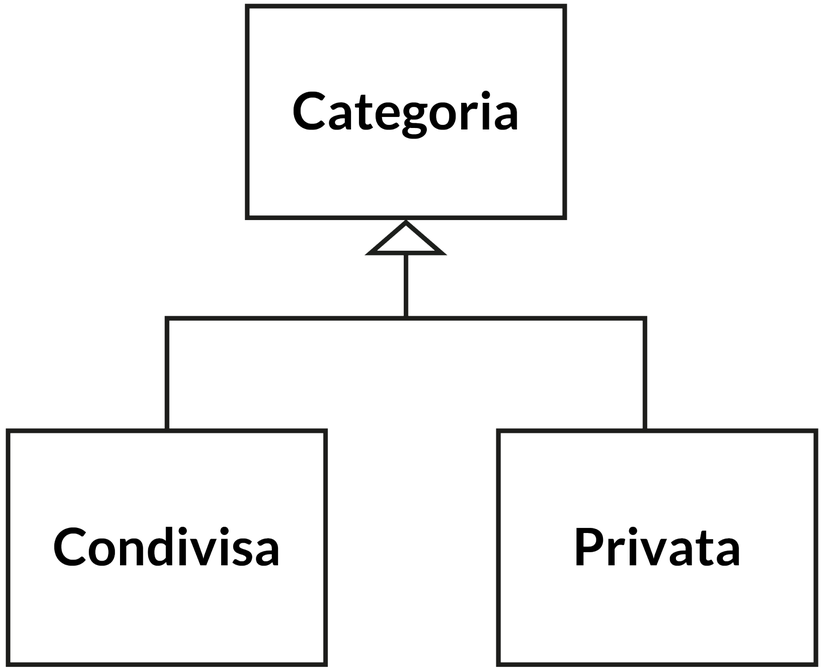
Invio degli scontrini



- Informazioni in JSON
- Chiamata POST
- Validazione dei dati
- Creazione scontrino “temporaneo”

DETTAGLI

Gestione delle categorie



- Categoria: superclasse astratta
- Propagazione delle modifiche diverse a seconda delle classi figlie
  - Notifica per le categorie condivise
  - Scrittura bufferizzata per le categorie provate

API

GOOGLE CLOUD FUNCTIONS

- Funzioni scritte in Javascript
- http request
- Macchine di diverse capacità
- 400 ms medi

```javascript
exports.sendScontrinoStandard = functions.https.onRequest( async (req, res) => {
  if(req.method === 'POST'){
    const infoS = req.body;
    let infoUser = await admin.firestore().collection(usersCollection).doc(infoS.uid).get();
    if(infoUser.exists === false) {
      return res.status(404).send({status: "Error", message: "Inexistent user"});
    }
    const token = infoUser.get(tokenFCM);
    if(token === null) {
      return res.status(404).send({status: "Error", message: "Inexistent token"});
    }
    if(typeof infoS[negozioScontrino] !== String || !Array.isArray(infoS[ricevutaScontrino])
      || typeof infoS[dataScontrino] !== String) {
      return res.status(400).send({status: "Error", message: "Wrong parameters"});
    }
    if(parseFloat(infoS[costoScontrino]) === NaN || parseFloat(infoS[areaScontrino]) === NaN) {
      return res.status(400).send({status: "Error", message: "Wrong parameters"});
    }
    let payload = {
      'data': {
        date: infoS[dataScontrino],
        negozio: infoS[negozioScontrino],
        area: infoS[areaScontrino],
        costo: infoS[costoScontrino],
        type: "sendScontrino",
        title: "Nuovo scontrino",
        click_action: "FLUTTER_NOTIFICATION_CLICK",
      },
      'notification': {
        title: "Nuovo scontrino",
        body: "Hai ricevuto un nuovo scontrino!",
      }
    }
```

Rules e test

FIREBASE RULES

- Per database e storage
- Controllo in gerarchie
- Diviso in *read*, *write*, *create* e *delete*
- Cloud armor

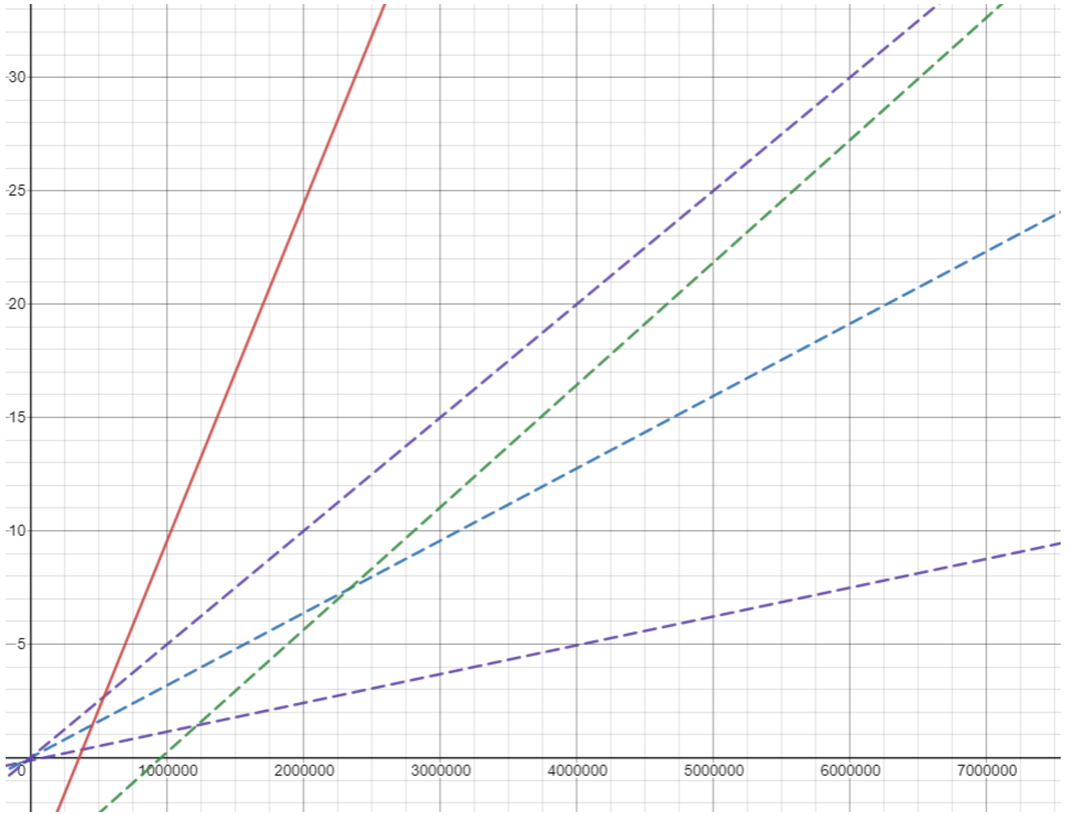
```
match /users/documents/{doc} {
    allow read: if request.auth != null;
}
```

TEST

- Unit test
- Widget test
- Integration test

```
21 void main() {
22   TestWidgetsFlutterBinding.ensureInitialized();
23
24   String email = "est@ee.it";
25   String username = "username";
26   String password = "123456";
27   String confPass = "123456";
28
29   setUpAll(() async {
30     await initHive();
31   });
32
33   group("Test sign page offline", () {
34     testWidgets("Pagina login statica", (WidgetTester tester) async {
35       await tester.pumpWidget(StartingPage(ScreenSignForm()));
36       expect(find.byType(FormTextInput), findsWidgets);
37       expect(find.text("Email"), findsWidgets);
38       expect(find.byType(ButtonGoogle), findsOneWidget);
39       expect(find.byKey(Key("Sign_button")), findsOneWidget);
40       expect(find.byType(ScreenSignForm), findsOneWidget);
41       expect(find.byType(MyHomePage), findsNothing);
42       expect(find.byKey(Key("Registrati_menu_button")), findsOneWidget);
43       expect(find.byKey(Key("Logo_payper")), findsOneWidget);
44     });
```

Conclusione



- Costo per scontrino digitale: **300 volte più economico**
- Valida e stabile alternativa all'utilizzo degli scontrini
- Previsti futuri colloqui con aziende

Grazie per l’attenzione